

EX. NO: 4**SINGLY LINKED LIST****DATE:****QUESTION :**

Implement creation of a sorted linked list, Deletion of a node, search a value, merge two sorted lists, and display operations on Singly Linked List

AIM:

To Implement creation of a sorted linked list, Deletion of a node, search a value, merge two sorted lists, and display operations on Singly Linked List

ALGORITHM:

Step 1: Start the program.

Step 2: Create a node structure with:

- Data field
- Next pointer

Step 3: Initialize the head pointer as NULL.

Step 4: Display the menu containing:

- Insert at Beginning
- Insert at End
- Insert at Position
- Delete from Beginning
- Delete from End
- Delete from Position
- Search
- Display
- Merge Two Lists
- Exit

Step 5: Read the user's choice.

Step 6: Perform the selected operation:

- Create a new node for insertion.
- Update links appropriately.
- Delete the required node and free memory.
- Search the element by traversing the list.
- Display all nodes.
- Merge the second list at the end of the first list.

Step 7: Repeat Steps 4–6 until the user selects Exit.

Step 8: Stop the program.

PROGRAM:

```

#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;
struct Node* createNode(int data)
{
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL)
    {
        printf("Memory Allocation Failed!\n");
        exit(1);
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
void insertBeginning(int data)
{
    struct Node *newNode = createNode(data);
    newNode->next = head;
    head = newNode;
}
void insertEnd(int data)
{
    struct Node *newNode = createNode(data);
    if (head == NULL)
    {
        head = newNode;
        return;
    }
    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
}
void insertPosition(int data, int pos)
{
    int i;

```

```
if (pos < 1)
{
    printf("Invalid Position\n");
    return;
}
if (pos == 1)
{
    insertBeginning(data);
    return;
}
struct Node *newNode = createNode(data);
struct Node *temp = head;
for (i = 1; i < pos - 1 && temp != NULL; i++)
    temp = temp->next;
if (temp == NULL)
{
    printf("Invalid Position\n");
    free(newNode);
    return;
}
newNode->next = temp->next;
temp->next = newNode;
}
void deleteBeginning()
{
    if (head == NULL)
    {
        printf("List is Empty\n");
        return;
    }
    struct Node *temp = head;
    head = head->next;
    free(temp);
}
void deleteEnd()
{
    if (head == NULL)
    {
        printf("List is Empty\n");
        return;
    }
    if (head->next == NULL)
```

```

    {
        free(head);
        head = NULL;
        return;
    }
    struct Node *temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;
    free(temp->next);
    temp->next = NULL;
}
void deletePosition(int pos)
{
    int i;
    if (head == NULL)
    {
        printf("List is Empty\n");
        return;
    }
    if (pos < 1)
    {
        printf("Invalid Position\n");
        return;
    }
    if (pos == 1)
    {
        deleteBeginning();
        return;
    }
    struct Node *temp = head;
    for (i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
    if (temp == NULL || temp->next == NULL)
    {
        printf("Invalid Position\n");
        return;
    }
    struct Node *del = temp->next;
    temp->next = del->next;
    free(del);
}
void search(int key)

```

```

{
    struct Node *temp = head;
    int pos = 1;
    while (temp != NULL)
    {
        if (temp->data == key)
        {
            printf("Element found at position %d\n", pos);
            return;
        }
        temp = temp->next;
        pos++;
    }
    printf("Element not found\n");
}

void display(struct Node *ptr)
{
    if (ptr == NULL)
    {
        printf("List is Empty\n");
        return;
    }
    while (ptr != NULL)
    {
        printf("%d -> ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

// Merge Lists
struct Node* mergeLists(struct Node *list1, struct Node *list2)
{
    if (list1 == NULL)
        return list2;
    struct Node *temp = list1;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = list2;
    return list1;
}

int main()
{

```

```
int choice, data, pos;
while (1)
{
    printf("\n--- Linked List Menu ---\n");
    printf("1. Insert at Beginning\n");
    printf("2. Insert at End\n");
    printf("3. Insert at Position\n");
    printf("4. Delete from Beginning\n");
    printf("5. Delete from End\n");
    printf("6. Delete from Position\n");
    printf("7. Search Element\n");
    printf("8. Display\n");
    printf("9. Merge Another List\n");
    printf("10. Exit\n");
    printf("Enter Choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1:
            printf("Enter Data: ");
            scanf("%d", &data);
            insertBeginning(data);
            break;
        case 2:
            printf("Enter Data: ");
            scanf("%d", &data);
            insertEnd(data);
            break;
        case 3:
            printf("Enter Data and Position: ");
            scanf("%d%d", &data, &pos);
            insertPosition(data, pos);
            break;
        case 4:
            deleteBeginning();
            break;
        case 5:
            deleteEnd();
            break;
        case 6:
            printf("Enter Position: ");
            scanf("%d", &pos);
```

```

        deletePosition(pos);
        break;
    case 7:
        printf("Enter Element to Search: ");
        scanf("%d", &data);
        search(data);
        break;
    case 8:
        display(head);
        break;
    case 9:
    {
        struct Node *head2 = NULL;
        struct Node *temp;
        int n, value, i;
        printf("Enter number of nodes in second list: ");
        scanf("%d", &n);
        for (i = 0; i < n; i++)
        {
            printf("Enter value: ");
            scanf("%d", &value);
            struct Node *newNode = createNode(value);
            if (head2 == NULL)
                head2 = newNode;
            else
            {
                temp = head2;
                while (temp->next != NULL)
                    temp = temp->next;
                temp->next = newNode;
            }
        }
        head = mergeLists(head, head2);
        printf("Lists Merged Successfully.\n");
        break; }
    case 10:
        exit(0);
    default:
        printf("Invalid Choice\n");
    } }
    return 0;
}

```

INPUT AND OUTPUT:

```
[ita2536@mritlin harshan]$ vi ex4.c
[ita2536@mritlin harshan]$ cc ex4.c
[ita2536@mritlin harshan]$ ./a.out
--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 1
Enter Data: 1

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 2
Enter Data: 2

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 3
Enter Data and Position: 3
3

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
```



```
Enter Choice: 1
Enter Data: 0

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 4

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 5

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 6
Enter Position: 2

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 8
1 -> NULL
```

```

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 7
Enter Element to Search: 1
Element found at position 1

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 9
Enter number of nodes in second list: 3
Enter value: 1
Enter value: 2
Enter value: 3
Lists Merged Successfully.

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 8
1 -> 1 -> 2 -> 3 -> NULL

```

```

--- Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Search Element
8. Display
9. Merge Another List
10. Exit
Enter Choice: 10
[ita2536@mritlin harshan]$

```

RESULT:

Thus the Implementation and creation of a sorted linked list, Deletion of a node, search a value, merge two sorted lists, and display operations on Singly Linked List was executed successfully.